What about using XML?

- To enable XML support, we need to additional configuration
- Add the required dependencies in the pom.xml (to enable automatic XML serialization support in Spring Boot)

```
<dependency>
   <groupId>com.fasterxml.jackson.dataformat</groupId>
   <artifactId>jackson-dataformat-xml</artifactId>
</dependency>
```

- Import the required packages

```
import com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlRootElement;
import com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlProperty;
```

26

What about using XML?

- Annotate the DTO with:
  - @JacksonXMLRootElement
    - It defines the root XML tag
  - @JacksonXMLProperty
    - It defines the XML fields names

esempio22

27

A quick summary

What happens	Spring solution
Accept header is set to application/json	Spring uses Jackson to convert in JSON
Accept header is set to application/xml	Spring uses Jackson XML to convert in XML
Accept header is missing	Spring picks the first converter based on internal registration order (usually JSON). If produces is used, Spring forces the format

- In the REST controller, use

```
produces = MediaType.*[TYPE]*
```

to specify the response type

```
APPLICATION_XML_VALUE
APPLICATION_JSON_VALUE
```

This overrides the suggestion of the content negotiation strategy and forces Spring to send the specified type

28

Using a request body to get data from clients #1

- We did some examples in which a client send data to a server
- We used (so many times!) a form and the POST method. Using POST data will be put in the body of the request:
  - By default, a browser automaticallty send the form data with the content type:

```
application/x-www-form-urlencoded
```

The controller reads each field of the form using @RequestParam

29

Using a request body to get data from clients #2

- In case the client wants to send a file (e.g. an image), some changes are needed. The content file to be specified into the form is:

```
multipart/form-data
```

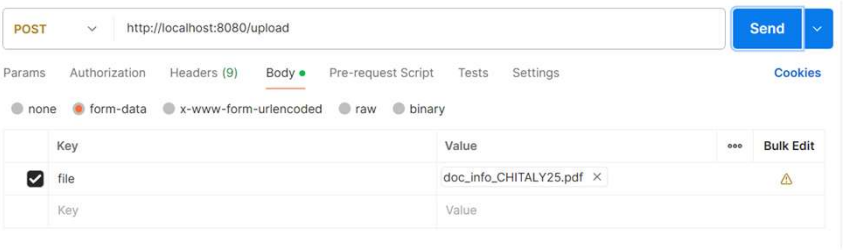
The controller will use the:
`@RequestParam("…") MultipartFile` annotation

30

Using a request body to get data from clients #2

esempio22bis

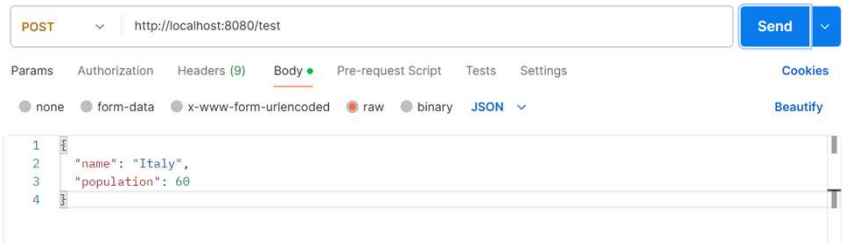
- Request HTTP (Postman / client)



31

Using a request body to get data from client #3

- What about sending a JSON?
- That can happen in several scenarios: a client using a JavaScript script and that wants to send something to a server; a server playing the role of a client and sending requests to another server
- The Controller will use @RequestBody to take data

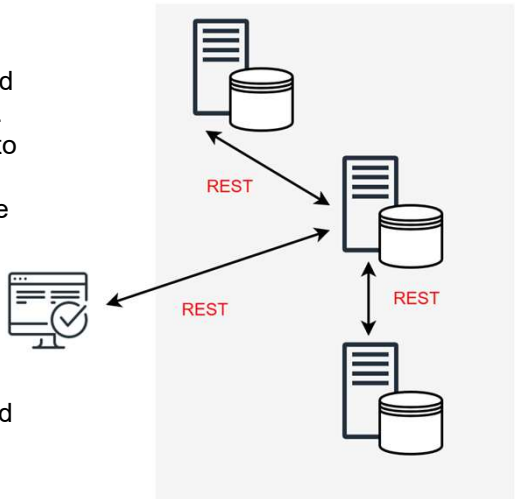


esempio23

32

REST services in Spring

- Back-end components communication
  - The back-end components implementing services need to speak to exchange data. So, we need to know how to call a REST endpoint exposed by another service
  - We will use OpenFeign, a tool offered by Spring Cloud



33

Adding Dependencies


```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
    <version>5.0.1</version>
</dependency>
```

34

How does OpenFeign work? - concept

- From a conceptual point of view:
  - It works on the server playing the role of a client
  - It prepares the request to be sent to the server:
    - If it needs to send data, it converts Java obects in JSON or XML
  - It receives data (e.g., JSON or XML) from the responses sent by the server and builds a Java obejct for such data

OpenFeign is really smart!
It avoids that developers become crazy in writing code

35

How does OpenFeign work? - code

- From an implementative point of view:
  - For each server the backend component wants to connect to, an interface is defined that contains methods, each corresponding to a possible request to the server (one action for each server's endpoint). The method's parameters specify what is sent in the request, the return value what came back with the response
  - OpenFeign automatically write the class implementing the interface
  - In the interface annotations are used to specify the server's URL and the endpoint (the action)

36

How does OpenFeign work? - code

- OpenFeign needs to know where to find the interfaces
- A Configuration class is needed to enable OpenFeign functionality and the search for interfaces. The annotation @EnableFeignClient is used.

```
@Configuration
@EnableFeignClients(basePackages = "*PACKAGE NAME*")
public class ProjectConfig {
}
```

LOOK HERE!

37

Now we are ready to go!

OpenFeign0A –OpenFeign0B

38

Exercise 1

- Si progetti e si implementi una web app che ottenga da un web service la data e l'ora corrente e la presenti a video. La web app mostra il seguente output:

Date and time example

Time zone: Europe/Rome

Today is Monday, April 14, 2025

Time is 17:15

Per ottenere le informazioni relative a data e ora, la web app interroga il servizio https://timeapi.io/, in particolare l'end-point api/time/current/zone.

39

Exercise 2

- Si estenda l'esercizio 1, ottenendo dal web service la lista di time zone disponibili (si può utilizzare l'end-point api/timezone/availabletimezones) e permettendo all'utente di selezionare la timezone voluta tramite una dropdown box. Il valore della time zone selezionata viene salvato per la durata dell'interazione dell'utente con la web app. La web app mostra il seguente output::

40